

Data Science Lab

Jupyter Notebooks – Code Analysis & Commentary

Notebooks 1 – 10 | Python · Pandas · Scikit-learn · NLTK · Seaborn

Black theme · Yellow code · Inline comments

1. Data Wrangling – I

Objective: Load the IRIS dataset, explore its structure, handle data types, apply Min-Max normalization, and encode categorical labels using Label Encoding and One-Hot Encoding.

■ Imports & Load

pandas and numpy are imported. The IRIS CSV is loaded into a DataFrame.

```
import pandas as pd
import numpy as np
df = pd.read_csv('IRIS.csv') # Load IRIS dataset into DataFrame
```

■ Basic Exploration

We inspect index, column names, shape, data types, and column values to understand the dataset structure.

```
df.index # Row index range
df.columns # Column labels
df.shape # (rows, cols) tuple
df.dtypes # Data type of each column
df.describe() # Statistical summary (mean, std, min, max, etc.)
```

■ Null Check

Check which columns contain missing values before any processing.

```
df.isnull().any() # Boolean: any nulls per column?
df.sepal_length.isnull().sum() # Count of nulls in sepal_length
```

■ Type Casting

petal_length is cast to integer to demonstrate dtype conversion.

```
df['petal_length'] = df['petal_length'].astype('int') # Cast float -> int
df.dtypes # Verify the change
```

■ Min-Max Normalization

sklearn's MinMaxScaler scales all numeric features to [0, 1] range, removing unit bias.

```
from sklearn import preprocessing
min_max_scaler = preprocessing.MinMaxScaler() # Scales to [0,1]
x = df.iloc[:, :4] # Select first 4 numeric cols
x_scaled = min_max_scaler.fit_transform(x) # Fit & transform
df_normalized = pd.DataFrame(x_scaled) # Wrap result in DataFrame
```

■ Label Encoding

Converts string species names to integer codes (0, 1, 2) so ML models can process them.

```
label_encoder = preprocessing.LabelEncoder()
df['species'] = label_encoder.fit_transform(df['species']) # e.g. 'Iris-setosa' -> 0
df['species'].unique() # Check encoded values
```

■ One-Hot Encoding

Creates binary columns per species class, avoiding ordinal bias introduced by label encoding.

```
encoder = preprocessing.OneHotEncoder()
encoded_species = encoder.fit_transform(df[['species']]).toarray() # Sparse -> dense
encoded_df = pd.DataFrame(encoded_species, columns=encoder.categories_[0])
```


2. Data Wrangling – II

Objective: Handle missing values in a student performance dataset using multiple imputation strategies (mean, median, std, min, max), detect outliers via boxplots, and apply log transformation.

■ Load & Null Detection

Load the CSV and identify rows/columns with null values.

```
df = pd.read_csv('StudentsPerformanceTest2.csv')
df.isnull() # Boolean DataFrame of all null positions
series = pd.isnull(df['math score']) # Boolean series for one column
df[series] # Rows where math score is null
```

■ Label Encode Gender

Encode the 'gender' column from strings to integers for downstream analysis.

```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
df['gender'] = le.fit_transform(df['gender']) # male/female -> 0/1
```

■ Custom NA Values & Fill

Re-read CSV treating 'Na/na' strings as NaN, then impute nulls per column strategy.

```
missing_values = ['Na', 'na']
df = pd.read_csv('StudentsPerformanceTest2.csv', na_values=missing_values)
# Impute math score with column MEAN
m_v = df['math score'].mean()
df['math score'].fillna(value=m_v, inplace=True)
# Impute reading score with MEDIAN
me_v = df['reading score'].median()
df['reading score'].fillna(value=me_v, inplace=True)
# Impute writing score with STD (unusual but demonstrates the method)
std_v = df['writing score'].std()
df['writing score'].fillna(value=std_v, inplace=True)
# Impute Placement Score with MIN
min_v = df['Placement Score'].min()
df['Placement Score'].fillna(value=min_v, inplace=True)
# Impute placement offer count with MAX
max_v = df['placement offer count'].max()
df['placement offer count'].fillna(value=max_v, inplace=True)
```

■ Outlier Detection – Boxplot

Visualise spread and outliers using boxplots for score columns.

```
col = ['math score', 'reading score', 'writing score', 'placement score']
df.boxplot(col) # Boxes show IQR; whiskers show outlier boundaries
```

■ Scatter & Log Transform

Inspect placement scores vs offer counts and apply log10 to reduce skew.

```
ax.scatter(df['placement score'], df['placement offer count'])
df['log_math'] = np.log10(df['math score']) # Log-transform to reduce right skew
df['log_math'].plot(kind='hist') # Compare distribution shape
```

3. Descriptive Statistics

Objective: Compute central tendency (mean, median, mode), spread (std, min, max) on a mall customers dataset, perform group-by analysis, and describe IRIS sub-populations by species.

■ Central Tendency

Mean, median, and mode summarise where data clusters.

```
df.mean(numeric_only=True) # Column-wise mean
df.loc[:, 'Age'].mean() # Mean of Age column only
df.median(numeric_only=True) # Column-wise median
df.mode() # Most frequent value per column
```

■ Spread Metrics

Standard deviation, min, and max show how data is distributed around the centre.

```
df.std(numeric_only=True) # Standard deviation (spread)
df.max() # Maximum values
df.min() # Minimum values
df.loc[:, 'Age'].std() # Std dev of Age specifically
```

■ GroupBy Analysis

Compute mean Age grouped by Genre to compare demographic groups.

```
df.groupby(['Genre'])['Age'].mean() # Average age per genre group
```

■ One-Hot Encoding on Categorical

Encode the Genre column for downstream use with sklearn.

```
enc = preprocessing.OneHotEncoder()
enc_df = pd.DataFrame(enc.fit_transform(df[['Genre']]).toarray())
df_encode = df_u.join(enc_df) # Append encoded cols to renamed DataFrame
```

■ Per-Species Description of IRIS

Filter IRIS dataset by species and call describe() to get statistics for each class independently.

```
irisSet = (iris['species'] == 'Iris-setosa')
print(iris[irisSet].describe()) # Stats only for setosa rows
irisVer = (iris['species'] == 'Iris-versicolor')
print(iris[irisVer].describe()) # Stats only for versicolor rows
```

4. Data Analysis I – Linear Regression

Objective: Implement simple polynomial fit on toy data, then build a full sklearn LinearRegression pipeline on the California Housing dataset with train/test split and evaluation metrics.

■ Simple Polynomial Fit

numpy polyfit finds best-fit line coefficients; poly1d wraps them for easy prediction.

```
x = np.array([95, 85, 80, 70, 60]) # Feature: study hours or scores
y = np.array([85, 95, 70, 65, 70]) # Target: result scores
model = np.polyfit(x, y, 1) # Degree-1 = linear: returns [slope, intercept]
predict = np.poly1d(model) # Create callable predictor
predict(65) # Predict for x=65
```

■ R2 Score on Simple Fit

R2 (coefficient of determination) measures how well predictions match actual values (1.0 = perfect).

```
from sklearn.metrics import r2_score
y_pred = predict(x) # Predictions for all training x
r2_score(y, y_pred) # R2 score: 1 = perfect, 0 = baseline
```

■ California Housing – Load & Split

Load a real-world housing dataset, drop the target to form X, then split 80/20.

```
from sklearn.datasets import fetch_california_housing
housing = fetch_california_housing()
data = pd.DataFrame(housing.data, columns=housing.feature_names)
data['PRICE'] = housing.target # Add target column
x = data.drop(['PRICE'], axis=1) # Features
y = data['PRICE'] # Target
X_train, X_test, y_train, y_test = train_test_split(x, y, test_size=0.2)
```

■ Train & Evaluate LinearRegression

Fit model on training set, predict on both sets, compute MSE to detect overfitting.

```
lm = LinearRegression()
lm.fit(X_train, y_train) # Learn weights from training data
y_train_pred = lm.predict(X_train) # Predict on train set
y_test_pred = lm.predict(X_test) # Predict on unseen test set
mse = mean_squared_error(y_test, y_test_pred) # Lower MSE = better
print(mse)
```

■ Scatter Plot – True vs Predicted

Visually compare actual vs predicted prices; ideal model lies along $y=x$ diagonal.

```
plt.scatter(y_train, y_train_pred, c='blue', label='Training data')
plt.scatter(y_test, y_test_pred, c='lightgreen', label='Test data')
plt.xlabel('True values') # Actual house price
plt.ylabel('Predicted') # Model-predicted price
plt.title('True value vs Predicted value')
plt.legend(loc='upper left')
plt.show()
```

5. Data Analysis II – Logistic Regression

Objective: Train a binary Logistic Regression classifier, evaluate it with a confusion matrix, and compute Accuracy, Precision, Recall, F1 Score, and Specificity.

■ Load & Preprocess

Drop the non-numeric 'Genre' column so only numeric features are passed to the model.

```
df = pd.read_csv('Dataset5.csv')
df1 = df.drop('Genre', axis=1) # Remove categorical column
y = df1.iloc[:, -1].values # Last column = target (0 or 1)
X = df1.iloc[:, :-1].values # All other columns = features
```

■ Train/Test Split

25% data held out for testing; random_state=2 ensures reproducibility.

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=2)
```

■ Fit Logistic Regression

LogisticRegression applies a sigmoid function to output class probabilities.

```
LR = LogisticRegression() # Default solver; finds decision boundary
LR.fit(X_train, y_train) # Learn from training data
y_pred = LR.predict(X_test) # Predict class labels (0 or 1) for test set
```

■ Confusion Matrix & Metrics

Unpack confusion matrix into TP, TN, FP, FN and calculate each metric manually.

```
tn, fp, fn, tp = confusion_matrix(y_test, y_pred).ravel()
# tn=correctly predicted negatives, fp=false alarms
# fn=missed positives, tp=correctly predicted positives
accuracy = (tn + tp) * 100 / (tp + tn + fp + fn) # % correct overall
precision = tp / (tp + fp) # Of predicted +ve, how many are really +ve
recall = tp / (tp + fn) # Of actual +ve, how many did we catch
f1_score = (2 * precision * recall) / (precision + recall) # Harmonic mean of P & R
specificity = tn / (tn + fp) # True Negative Rate
```

■ User Input Prediction

Loop over feature count, collect values, reshape to (1, n_features) and predict.

```
for i in range(X_train.shape[1]):
    val = float(input(f'Feature {i+1}: '))
    input_features.append(val)
input_array = np.array(input_features).reshape(1, -1) # Model expects 2D
prediction = LR.predict(input_array)[0] # Returns 0 or 1
print(f'Predicted Output: {prediction}')
```

6. Data Analysis III – Naive Bayes

Objective: Build a Gaussian Naive Bayes classifier, evaluate with accuracy/precision/recall, display the confusion matrix, scale features, and accept user input for real-time prediction.

■ Load & Split

Load dataset, select all-but-last as features and last as target, split 75/25.

```
dataset = pd.read_csv('Dataset6.csv')
X = dataset.iloc[:, :-1].values # Features: all columns except last
y = dataset.iloc[:, -1].values # Target: last column
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=2)
```

■ Gaussian Naive Bayes

Assumes each feature follows a Gaussian distribution within each class; calculates posterior probabilities.

```
from sklearn.naive_bayes import GaussianNB
classifier = GaussianNB() # Assumes Gaussian likelihood P(x|class)
classifier.fit(X_train, y_train) # Estimate mean & variance per feature per class
y_pred = classifier.predict(X_test) # Apply Bayes theorem to classify
```

■ Evaluation Metrics

'micro' averaging sums TP/FP/FN across all classes before computing ratios.

```
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='micro') # Pooled across classes
recall = recall_score(y_test, y_pred, average='micro')
cm = confusion_matrix(y_test, y_pred) # Matrix rows=actual, cols=predicted
```

■ Feature Scaling + User Input

StandardScaler is applied AFTER training metrics so the scaled model is used only for prediction.

```
sc = StandardScaler()
X_train = sc.fit_transform(X_train) # Fit on train, transform train
X_test = sc.transform(X_test) # Only transform test (no data leakage)
age = float(input('Enter age: '))
salary = float(input('Enter salary: '))
scaled_input = sc.transform([[age, salary]]) # Must scale input same as training
prediction = classifier.predict(scaled_input)
```


7. Text Analytics

Objective: Perform NLP pipeline: sentence/word tokenization, stopwords removal, stemming, lemmatization, POS tagging, and sentiment classification using CountVectorizer and TF-IDF with Naive Bayes.

■ Tokenization

Split text into sentences and then individual words using NLTK.

```
from nltk.tokenize import sent_tokenize, word_tokenize
tokenized_text = sent_tokenize(text) # Split at '.', '?', '!'
tokenized_word = word_tokenize(text) # Split at spaces and punctuation
```

■ Frequency Distribution

Count how often each word/token appears in the text.

```
from nltk.probability import FreqDist
fdist = FreqDist(tokenized_word) # Token -> frequency mapping
```

■ Stopword Removal

Remove common words (the, is, and...) that carry little semantic meaning.

```
stop_words = set(stopwords.words('english')) # 179 English stopwords
filtered_sent = [w for w in tokenized_word if w not in stop_words]
```

■ Stemming vs Lemmatization

Stemming chops word endings; Lemmatization maps to root dictionary form.

```
ps = PorterStemmer()
stemmed = [ps.stem(w) for w in filtered_sent] # 'running' -> 'run', crude
lem = WordNetLemmatizer()
lem.lemmatize('flying', 'v') # 'flying' -> 'fly' (verb form), context-aware
```

■ POS Tagging

Labels each token with its grammatical role (NN, VB, JJ, etc.).

```
nltk.download('averaged_perceptron_tagger_eng')
nltk.pos_tag(tokens) # e.g. [('Einstein', 'NNP'), ('was', 'VBD'), ...]
```

■ CountVectorizer + Naive Bayes

Transform text to bag-of-words frequency matrix, then classify sentiments.

```
from sklearn.feature_extraction.text import CountVectorizer
cv = CountVectorizer(lowercase=True, stop_words='english', ngram_range=(1,1))
text_counts = cv.fit_transform(data['Phrase']) # Sparse term-frequency matrix
clf = MultinomialNB().fit(X_train, y_train) # Train on frequency features
print('MultinomialNB Accuracy:', metrics.accuracy_score(y_test, predicted))
```

■ TF-IDF Vectorizer

TF-IDF down-weights very common words and up-weights discriminating terms.

```
tf = TfidfVectorizer() # TF * log(N/df) weighting
text_tf = tf.fit_transform(data['Phrase']) # Sparse TF-IDF matrix
# Retrain Naive Bayes on TF-IDF features
clf = MultinomialNB().fit(X_train, y_train)
print('MultinomialNB Accuracy:', metrics.accuracy_score(y_test, predicted))
```

8. Data Visualisation – I

Objective: Explore the Titanic dataset visually using Seaborn: distribution plots, joint plots, pairplots, bar charts, box plots, violin plots, strip plots, and histplots.

■ Load & Inspect

Load Titanic training data and check structure before plotting.

```
dataset = pd.read_csv('Train8.csv')
dataset.info() # Column types and non-null counts
dataset.describe() # Summary statistics
dataset.isnull().sum() # Count missing per column
```

■ Distribution Plots

distplot shows histogram + KDE overlay to visualise Fare distribution.

```
sns.distplot(dataset['Fare']) # Histogram + KDE
sns.distplot(dataset['Fare'], kde=False) # Histogram only
sns.distplot(dataset['Fare'], kde=False, bins=10) # Control bin count
```

■ Relational & Pairwise Plots

jointplot shows bivariate distribution; pairplot shows all feature pairs.

```
sns.jointplot(x='Age', y='Fare', data=dataset) # Scatter + marginal hists
sns.pairplot(dataset) # All column combinations
```

■ Bar, Count & Rug Plots

Bar aggregates y by x; countplot counts occurrences; rugplot shows 1D density.

```
sns.barplot(x='Sex', y='Fare', data=dataset) # Mean Fare by Sex
sns.barplot(x='Sex', y='Age', data=dataset, estimator=np.std) # Std of Age
sns.countplot(x='Fare', data=dataset) # Count of each Fare value
sns.rugplot(dataset['Fare']) # Tick mark per observation
```

■ Box, Violin & Strip Plots

Compare distributions across categories with different levels of detail.

```
sns.boxplot(x='Sex', y='Age', data=dataset) # IQR boxes by sex
sns.boxplot(x='Survived', y='Fare', data=dataset) # Fare by survival
sns.boxplot(x='Sex', y='Age', data=dataset, hue='Survived') # Nested grouping
sns.violinplot(x='Sex', y='Age', data=dataset, hue='Survived') # KDE shape
sns.stripplot(x='Sex', y='Age', data=dataset) # All data points
sns.histplot(data=dataset, x='Fare', y='Survived') # 2D histogram
```

9 & 10. Data Visualisation – II & III

Objective: Continue Titanic visualisations with grouped box plots and extend to IRIS dataset using 2x2 histogram grids, multiple boxplots, and Seaborn boxplot comparisons.

■ Visualisation II – Titanic Boxplots

Repeated grouped boxplots to compare age/fare distributions by sex and survival.

```
sns.boxplot(x='Sex', y='Age', data=dataset) # Age by sex
sns.boxplot(x='Sex', y='Age', data=dataset, hue='Survived') # + survival
sns.boxplot(x='Survived', y='Fare', data=dataset) # Fare by survival
```

■ Visualisation III – IRIS 2x2 Histogram Grid

2x2 subplot layout to see distribution of each IRIS numeric feature.

```
df = pd.read_csv('Iris.csv')
df.columns = ['col1', 'col2', 'col3', 'col4', 'col5'] # Rename for brevity
fig, axes = plt.subplots(2, 2, figsize=(16, 8)) # 4 subplots in grid
axes[0,0].hist(df['col1']) # Sepal length distribution
axes[0,1].hist(df['col2']) # Sepal width distribution
axes[1,0].hist(df['col3']) # Petal length distribution
axes[1,1].hist(df['col4']) # Petal width distribution
```

■ IRIS Boxplots

Boxplot all four numeric features side by side using a list of Series.

```
data_to_plot = [df['col1'], df['col2'], df['col3'], df['col4']]
bp = ax.boxplot(data_to_plot) # Outliers visible as individual points
sns.boxplot(x='col1', y='col2', data=df) # Sepal length vs width
sns.boxplot(x='col3', y='col4', data=df) # Petal length vs width
```

Summary Table – All Notebooks

Notebook	Key Library	Core Technique	Output / Goal
1. Data Wrangling I	sklearn.preprocessing	MinMaxScaler, LabelEncoder, OHE	Normalised + encoded IRIS
2. Data Wrangling II	pandas	fillna (mean/median/std/min/max)	Clean student perf. dataset
3. Descriptive Stats	pandas, sklearn	mean/median/mode/std, groupby	Statistical summaries
4. Linear Regression	sklearn.linear_model	LinearRegression, polyfit	California housing MSE/R2
5. Logistic Regression	sklearn.linear_model	LogisticRegression, ConfMatrix	Accuracy 96%, F1=0.75
6. Naive Bayes	sklearn.naive_bayes	GaussianNB, StandardScaler	Multi-class classifier
7. Text Analytics	nltk, sklearn	Tokenize, TF-IDF, MultinomialNB	Sentiment classification
8. Visualisation I	seaborn, matplotlib	distplot, boxplot, violinplot	Titanic EDA plots
9. Visualisation II	seaborn	Grouped boxplots	Titanic survival plots
10. Visualisation III	matplotlib, seaborn	Subplot grid, hist, boxplot	IRIS feature distributions